NAME:K.Harsha Vardhan Reddy

REG NO:192425228

CO4-AT1

1) **Question 1: Bayesian Network for Disease Prediction**

A healthcare organization wants to predict whether a patient has Diabetes based on the symptoms Obesity and High Blood Sugar.

Coding Challenge:

Write a Python program to:

• Construct a Bayesian Network using the given variables.

• Define suitable Conditional Probability Tables (CPTs).

• Perform probabilistic inference to predict the probability of Diabetes when both symptoms are present.

• Display the predicted probability.

CODE :

```
 # Bayesian Network for Diabetes Prediction
import pgmpy
print("pgmpy is working")
print("Program started")
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Create Bayesian Network
model = DiscreteBayesianNetwork([
    ('Obesity', 'Diabetes'),
    ('High_Blood_Sugar', 'Diabetes')
])
# CPT for Obesity
cpd_obesity = TabularCPD(
    variable='Obesity',
    variable_card=2,
    values=[[0.7], [0.3]]
```

```
)

# CPT for High Blood Sugar
cpd_hbs = TabularCPD(
    variable='High_Blood_Sugar',
    variable_card=2,
    values=[[0.6], [0.4]]
)
# CPT for Diabetes
cpd_diabetes = TabularCPD(
    variable='Diabetes',
    variable_card=2,
    values=[
        [0.99, 0.80, 0.70, 0.10],  # No Diabetes
        [0.01, 0.20, 0.30, 0.90]   # Diabetes
    ],
    evidence=['Obesity', 'High_Blood_Sugar'],
    evidence_card=[2, 2]
)
# Add CPTs
model.add_cpds(cpd_obesity, cpd_hbs, cpd_diabetes)

# Check model
print("Model Valid:", model.check_model())
# User Input
obesity = int(input("Enter Obesity (0=No, 1=Yes): "))
hbs = int(input("Enter High Blood Sugar (0=No, 1=Yes): "))
# Inference
inference = VariableElimination(model)
result = inference.query(
    variables=['Diabetes'],
    evidence={
```

```
        'Obesity': obesity,
        'High_Blood_Sugar': hbs
    }
)
print("\nPrediction Result:")
print(result)
print("\nProbability of Diabetes =", round(result.values[1] * 100, 2), "%")
```

OUTPUT :

```
Model Valid: True
Enter Obesity (0=No, 1=Yes): 1
1
Enter High Blood Sugar (0=No, 1=Yes):
Prediction Result:
+-------------+-----------------+
| Diabetes    |   phi(Diabetes) |
+=============+=================+
| Diabetes(0) |          0.1000 |
+-------------+-----------------+
| Diabetes(1) |          0.9000 |
+-------------+-----------------+

Probability of Diabetes = 90.0 %
```

2) Question 2: Hidden Markov Model for Weather Prediction

A weather forecasting agency records daily observations as Sunny, Cloudy, and Rainy. The actual weather conditions are hidden and need to be predicted.

Coding Challenge:

Write a Python program to:

• Implement a Hidden Markov Model (HMM).

• Define transition and emission probabilities.

• Predict the hidden weather states for a given sequence of observations.

• Print the predicted sequence of hidden states.

CODE :

```
states = ["Sunny", "Cloudy", "Rainy"]
transition = {
    "Sunny": {"Sunny": 0.6, "Cloudy": 0.3, "Rainy": 0.1},
    "Cloudy": {"Sunny": 0.3, "Cloudy": 0.4, "Rainy": 0.3},
    "Rainy": {"Sunny": 0.1, "Cloudy": 0.4, "Rainy": 0.5}
}
emission = {
    "Sunny": {"Dry": 0.8, "Humid": 0.15, "Wet": 0.05},
    "Cloudy": {"Dry": 0.2, "Humid": 0.6, "Wet": 0.2},
    "Rainy": {"Dry": 0.05, "Humid": 0.25, "Wet": 0.7}
}
initial = {
    "Sunny": 0.5,
    "Cloudy": 0.3,
    "Rainy": 0.2
}
observations = ["Dry", "Humid", "Wet", "Wet"]
viterbi = [{}]
path = {}
```

```
for state in states:
    viterbi[0][state] = initial[state] * emission[state][observations[0]]
    path[state] = [state]
for t in range(1, len(observations)):
    viterbi.append({})
    new_path = {}
    for current in states:
        probability, previous = max(
            (
 viterbi[t - 1][prev] *
                transition[prev][current] *
                emission[current][observations[t]],
                prev
            )
            for prev in states
        )
        viterbi[t][current] = probability
        new_path[current] = path[previous] + [current]
    path = new_path
probability, final_state = max(
    (viterbi[-1][state], state)
    for state in states
)
print("HIDDEN MARKOV MODEL - WEATHER PREDICTION")
print("\nINPUT:")
print("Observations:", observations)
print("\nOUTPUT:")
print("Predicted Hidden Weather States:", path[final_state])
print("Probability:", round(probability, 6))
```

OUTPUT :

```
HIDDEN MARKOV MODEL - WEATHER PREDICTION

INPUT:
Observations: ['Dry', 'Humid', 'Wet', 'Wet']

OUTPUT:
Predicted Hidden Weather States: ['Sunny', 'Cloudy', 'Rainy', 'Rainy']
Probability: 0.005292
```

3) Question 3: Markov Random Field for Image Denoising

A computer vision company wants to remove noise from grayscale images using a Markov Random Field (MRF).

Coding Challenge:

Write a Python program to:

• Represent a small image as a graph where each pixel is a node.

• Model neighboring pixel relationships using an undirected graph.

• Simulate one iteration of image smoothing based on neighboring pixel values.

• Display the original and updated pixel values.

CODE :

```
image = [
    [100, 100, 100],
    [100, 250, 100],
    [100, 100, 100]
]
rows = len(image)
cols = len(image[0])
updated = [row[:] for row in image]
for i in range(rows):
    for j in range(cols):
        neighbors = []
        if i > 0:
            neighbors.append(image[i - 1][j])
        if i < rows - 1:
            neighbors.append(image[i + 1][j])
        if j > 0:
            neighbors.append(image[i][j - 1])
        if j < cols - 1:
```

```
            neighbors.append(image[i][j + 1])
        updated[i][j] = round(sum(neighbors) / len(neighbors))
print("MARKOV RANDOM FIELD - IMAGE DENOISING")
print("\nINPUT - Original Image:")
for row in image:
    print(row)
print("\nOUTPUT - Updated Image:")
for row in updated:
    print(row)
```

OUTPUT :

```
MARKOV RANDOM FIELD - IMAGE DENOISING

INPUT - Original Image:
[100, 100, 100]
[100, 250, 100]
[100, 100, 100]

OUTPUT - Updated Image:
[100, 150, 100]
[150, 100, 150]
[100, 150, 100]
>
```

4) Question 4: Conditional Random Field for Named Entity Recognition

A customer support system needs to identify Customer Names, Order IDs, and Product Names from support emails.

Coding Challenge:

Write a Python program to:

• Create sequential training data with word-level features.

• Train a Conditional Random Field (CRF) model.

• Predict entity labels for a new sentence.

• Display the predicted labels.

CODE :

```
from sklearn_crfsuite import CRF
train_data = [
  [
    ("John", "CUSTOMER"),
    ("ordered", "O"),
    ("Laptop", "PRODUCT"),
    ("Order123", "ORDER_ID")
  ],
  [
    ("Priya", "CUSTOMER"),
    ("bought", "O"),
    ("Phone", "PRODUCT"),
    ("Order456", "ORDER_ID")
  ],
  [
    ("Rahul", "CUSTOMER"),
    ("purchased", "O"),
    ("Headphones", "PRODUCT"),
```

```
        ("Order789", "ORDER_ID")
    ]
]
def features(sentence, i):
    word = sentence[i]
    return {
        "word": word,
        "lower": word.lower(),
        "first_letter": word[0],
        "is_digit": word.isdigit(),
        "length": len(word)
    }
X_train = []
y_train = []
for sentence in train_data:
    words = [x[0] for x in sentence]
    labels = [x[1] for x in sentence]
    X_train.append([features(words, i) for i in range(len(words))])
    y_train.append(labels)
crf = CRF(algorithm="lbfgs", max_iterations=100)
crf.fit(X_train, y_train)
sentence = input("Enter support sentence: ")
words = sentence.split()
X_test = [[features(words, i) for i in range(len(words))]]
prediction = crf.predict(X_test)[0]
print("\nPredicted Labels:")
for word, label in zip(words, prediction):
    print(word, "->", label)
```

OUTPUT :

```
Enter support sentence: John ordered Laptop Order123
```

```
Predicted Labels:
John -> CUSTOMER
ordered -> O
Laptop -> PRODUCT
Order123 -> ORDER_ID
```

5) Question 5: Bayesian Network Learning from Data

An insurance company wants to learn probabilistic relationships among Age, Income, Vehicle Type, and Insurance Claim using historical customer data.

Coding Challenge:

Write a Python program to:

• Load a dataset using Pandas.

• Learn the Bayesian Network structure from the data.

• Perform inference to predict the probability of an insurance claim for a new customer.

• Display the learned network structure and prediction results.

CODE :

```
import pandas as pd
import warnings
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination
warnings.filterwarnings("ignore", category=FutureWarning)
data = pd.DataFrame({
    "Age": ["Young","Young","Middle","Middle","Old","Old","Young","Middle","Old","Young"],
    "Income": ["Low","High","Low","High","Low","High","High","High","Low","Low"],
    "Vehicle": ["Car","Bike","Car","Car","Bike","Car","Bike","Bike","Car","Bike"],
    "Claim": ["No","No","Yes","No","Yes","No","Yes","No","Yes","Yes"]
})
print("INPUT DATA:")
print(data)
model = DiscreteBayesianNetwork([
    ("Age", "Claim"),
    ("Income", "Claim"),
```

```
    ("Vehicle", "Claim")
])
estimator = MaximumLikelihoodEstimator(model, data)
model.add_cpds(*estimator.get_parameters())
print("\nLEARNED NETWORK STRUCTURE:")
print(list(model.edges()))
age = input("\nEnter Age (Young/Middle/Old): ").strip().capitalize()
income = input("Enter Income (Low/High): ").strip().capitalize()
vehicle = input("Enter Vehicle (Car/Bike): ").strip().capitalize()
if age not in ["Young", "Middle", "Old"]:
    print("Invalid Age")
    exit()
if income not in ["Low", "High"]:
    print("Invalid Income")
    exit()
if vehicle not in ["Car", "Bike"]:
    print("Invalid Vehicle")
    exit()
inference = VariableElimination(model)
result = inference.query(
    variables=["Claim"],
    evidence={
        "Age": age,
        "Income": income,
        "Vehicle": vehicle
    }
)
print("\nOUTPUT:")
print("Probability of No Claim =", round(result.values[0] * 100, 2), "%")
print("Probability of Claim =", round(result.values[1] * 100, 2), "%")
```

OUTPUT:

```
INPUT DATA:
      Age Income Vehicle Claim
0   Young    Low     Car    No
1   Young   High    Bike    No
2  Middle    Low     Car   Yes
3  Middle   High     Car    No
4     Old    Low    Bike   Yes
5     Old   High     Car    No
6   Young   High    Bike   Yes
7  Middle   High    Bike    No
8     Old    Low     Car   Yes
9   Young    Low    Bike   Yes

LEARNED NETWORK STRUCTURE:
[('Age', 'Claim'), ('Income', 'Claim'), ('Vehicle', 'Claim')]

Enter Age (Young/Middle/Old): young
Enter Income (Low/High): high
Enter Vehicle (Car/Bike): bike

OUTPUT:
Probability of No Claim = 50.0 %
Probability of Claim = 50.0 %
```